Exact Parallel Propagation and Throttling with One Leak on a Cycle

Ryutaro Yonezu
Yonezu Research Lab
yonezaemon1+research@gmail.com

26 October 2026 | Version 1.0
Research Note No.41 | Unrefereed preprint

Abstract

Consider standard zero forcing on a simple undirected cycle, with simultaneous rounds and one fixed adversarial vertex that cannot force but may become blue. For an arbitrary initial set, retain its maximal blue runs of length at least two. The worst completion time is the largest number of initially white vertices in an arc between consecutive retained runs; isolated initial blue vertices do not split such an arc. We give an elementary proof using the known one-leaky robustness of path endpoints, a time-preserving removal of initial blue–blue edges, and an explicit lower-bound witness. Balanced layouts then give the exact seed-budget frontier, deadline minimum, and additive throttling value $\lceil\sqrt{8n}\rceil - 2$. The established forcing model and propagation mechanism are not new. This note makes an exact time formula explicit; historical novelty remains unestablished.

1 Scope and relation to established results

Leaky forcing, ordinary propagation time, and throttling are established notions. Alameda, Dillman and Kenter prove that the endpoints of a path form a one-leaky forcing set [1, Proposition 3.1, p.312]. The same fact follows by specializing the tree theorem of Alameda, Kritschgau, Warnberg and Young [2, Theorem 4.4] to one leak. The usual cycle robustness argument uses two opposing fronts [1, Proposition 3.4, p.313]. We use these facts rather than claim a new fault-bypass mechanism.

The upper time count below uses the familiar principle that a successful process must make progress at every step. Hogben et al. use this principle for an ordinary propagation-time bound [3, Remark 1.8]. Here its application to an arbitrary initial set and a fixed leak is justified explicitly. Butler and Young develop the forcing-chain capacity and path construction behind standard throttling [4, Section 2, pp.66–68]. The arithmetic optimization in this note is a consequence of the fixed-set formula, not an independent optimization principle.

Carlson et al. determine ordinary cycle throttling and study PSD and weighted PSD throttling [5, Sections 6–7]. Their color-change rules must be distinguished from the fixed vertex-leak rule used here. In particular, standard forcing does not let an isolated initial blue vertex start a front while both of its neighbors remain white.

The intended scope is an explicit exact time formula and an elementary reproof using known path facts plus the supplementary observations proved below. No priority, new forcing model, human peer review, or complete literature search is claimed. An elementary derivation does not establish that the same time formula was previously published. This is a mathematical graph model, without a claim about measured physical transmission time.

2 Fixed model and notation

Let C_n be the simple undirected cycle on $\mathbb{Z}/n\mathbb{Z}$, $n \geq 3$, with edges $\{i, i+1\}$. First choose an initial blue set B. An adversary then chooses one fixed vertex x from *all* vertices, including initially white ones. Vertex x cannot force, but can itself become blue. No vertex or edge is deleted in the model.

Time zero is the initial coloring. In each round, every blue vertex other than x with exactly one white neighbor forces that neighbor blue. All decisions use the coloring at the *start* of the round and are applied simultaneously. A newly blue vertex can force only in a later round. There is no domination step, hopping, PSD component rule, or moving leak.

Let $p_G(B; x)$ be the first round at which all vertices of G are blue, or ∞ if the process stalls, and set

$$\begin{aligned} T_G(B) &= \max_{x \in V(G)} p_G(B; x), \\ \tau(n, k) &= \min_{|B|=k} T_{C_n}(B), \\ m(n, t) &= \min\{|B| : T_{C_n}(B) \leq t\}, \\ \mathrm{th}_1(C_n) &= \min_B (|B| + T_{C_n}(B)). \end{aligned}$$

Here t is a nonnegative integer, and infinite-time sets do not improve the last minimum. We write $T(B) = T_{C_n}(B)$ when clear. For $k = n$ the time is zero; for $k = 0, 1$ it is infinite. The notation th_1 is local to this note.

For a fixed graph and leak, enlarging the initial blue set preserves the round-by-round inclusion of blue sets. Indeed, a force $u \to v$ valid for a smaller blue set is either already satisfied in the larger set or remains valid because u and all its other neighbors are blue there. This also shows that applying all available forces is at least as fast as any legal schedule.

3 The fixed-set formula

For a proper initial set B, let $R_1, \ldots, R_r$ be its maximal initial blue runs of length at least two, in cyclic order. For $r > 0$, let I_i be the arc strictly between R_i and R_{i+1}, with cyclic indices. Let a_i be the last vertex of R_i before the arc and b_i the first vertex of R_{i+1} after it. These endpoints are distinct even when $r = 1$.

The arc begins and ends with white vertices. Any initial blue vertices inside it are isolated. Let W_i count the *initially white* vertices in I_i. Thus $W_i \geq 1$ and

$$\sum_i W_i = n - |B|. \tag{1}$$

Its length may exceed W_i.

Observation O1 (initial blue–blue edge removal). *Let F be any collection of edges of a graph G whose endpoints are both initially blue, and put $H = G - F$. Keep every vertex, the initial set, and the fixed leak unchanged. Then the blue set at every round is identical in G and H. In particular $p_H(B; x) = p_G(B; x)$ for every x.*

Proof. For any current blue set $D \supseteq B$ and any vertex v,

$$N_H(v) \setminus D = N_G(v) \setminus D.$$

Every removed neighbor is in B and remains blue. No removed edge can be used to force a white target. Hence the possible forces, including the restriction at x, are the same. Induction from time zero proves the claim. □

Theorem A (fixed-set worst time). *If $B = V(C_n)$ then $T(B) = 0$. If B is proper with no adjacent initial blue vertices then $T(B) = \infty$. In all other cases,*

$$T(B) = \max_i W_i. \tag{A}$$

Placing the leak at b_j for any arc with maximum W_j attains this time.

Proof. The full-set case is immediate. With no adjacent initial blue vertices, every blue vertex has two white neighbors, so no force starts, even without a leak.

Otherwise apply O1 to all initial blue–blue edges:

$$H = C_n - E(C_n[B]).$$

The components containing white vertices are disjoint paths $P_i = a_i, I_i, b_i$, with both endpoints initially blue and only isolated initial blue vertices inside. Other components are isolated blue vertices. This description also covers a run of length two and the case of one long run. It preserves the original W_i. The edge removal is an analytical transformation under O1, not a change to the failure model.

For the upper bound, the two endpoints of each P_i form a one-leaky forcing set by the known path result [1, Proposition 3.1], or the tree theorem [2, Theorem 4.4]. Adding the internal initial blue vertices preserves success by monotonicity. If the global leak is outside the component, its process has no leak and succeeds as well.

A successful fixed-leak process cannot have a round with no color change before completion. Such a round would leave the same coloring and the same prohibition, causing permanent stalling. Each incomplete round therefore colors at least one of the W_i white vertices. Thus P_i finishes within W_i rounds. The components run in parallel, and O1 gives $T(B) \le \max_i W_i$.

For the lower bound, choose P_j with maximum W_j and leak its endpoint b_j. List its initially white vertices from a_j to b_j as $w_1, \dots, w_{W_j}$. We show that at time q, $0 \le q \le W_j$, precisely $w_1, \dots, w_q$ have become blue. This is true at time zero. If $q < W_j$, the vertex immediately before w_{q+1} is a nonleaky blue vertex: it is a_j, w_q, or the initial blue singleton between w_q and w_{q+1}. Its backward neighbor, if present, is blue. It can therefore force w_{q+1} in the next round.

Every initial blue singleton further along still has two white neighbors at the start of that round. In particular, the singleton just beyond w_{q+1} cannot simultaneously force the following white vertex: w_{q+1} is still white when decisions are made. Newly colored vertices cannot force within the same round, and entry from b_j is prohibited. Thus no other white target is reached. This proves the induction. The component needs exactly W_j rounds, and O1 gives $T(B) \ge W_j$. □

O1 and the lower-bound count are supplementary elementary arguments proved here. The cited reachability theorem does not itself state (A).

4 Seed budgets and deadlines

Theorem B (fixed-budget optimum). *For $n \ge 3$ and $2 \le k < n$,*

$$\tau(n, k) = \left\lceil \frac{n-k}{\lfloor k/2 \rfloor} \right\rceil. \tag{B}$$

The remaining values are $\tau(n, n) = 0$ and $\tau(n, 0) = \tau(n, 1) = \infty$.

Proof. For a finite-time proper set of size k, every long blue run contains at least two seeds, so $r \le \lfloor k/2 \rfloor$. Equations (A) and (1) give

$$n - k = \sum_i W_i \le rT(B) \le \lfloor k/2 \rfloor T(B),$$

which proves the lower bound, including sets with isolated seeds.

For the upper bound, put $W = n - k > 0$ and $r = \min\{\lfloor k/2 \rfloor, W\}$. Form r blue runs of at least two seeds each, put all $k - 2r$ extra seeds in the first run, and separate them by r nonempty white runs whose lengths sum to W and differ by at most one. There are no isolated blue seeds in this layout. If $W \ge \lfloor k/2 \rfloor$, (A) gives the desired ceiling. If $W < \lfloor k/2 \rfloor$, each white run has one vertex and the time is one, also the value in (B). □

Corollary B1 (deadline certificate). *For an integer $t \ge 0$, a proper B completes by t against every fixed leak if and only if it has a long initial blue run and each intervening arc contains at most t initially white vertices. The full set completes at zero.*

Proof. This is exactly (A), including its stalled case. □

Corollary B2 (minimum seeds at a deadline). *For $n \ge 3$ and an integer $t \ge 0$,*

$$m(n,t) = \min\left\{2\left\lceil \frac{n}{t+2} \right\rceil,\ 2\left\lceil \frac{n-1}{t+2} \right\rceil + 1\right\}. \tag{C}$$

In particular $m(n,0) = n$, and $m(n,t) = 2$ if and only if $t \ge n - 2$.

Proof. By (B), a budget $k \ge 2$ is feasible precisely when $n \le k + t\lfloor k/2 \rfloor$, including $k = n$. For even $k = 2r$, the least candidate is $2\lceil n/(t+2) \rceil$; for odd $k = 2r+1$, it is $2\lceil (n-1)/(t+2) \rceil + 1$. A candidate above n cannot win: for $t = 0$ one parity gives exactly n, and for $t \ge 1$ the even candidate is at most n. The odd candidate is at least three and does not introduce $k = 1$. Finally $k = 0, 1$ are infeasible and $\tau(n,2) = n - 2$. □

5 Additive throttling

Theorem C (additive throttling). *For all $n \ge 3$,*

$$\mathrm{th}_1(C_n) = \left\lceil \sqrt{8n} \right\rceil - 2. \tag{D}$$

Proof. Let c be an integer cost budget, k the seed count, and t the time. Feasibility with $k + t \le c$ implies

$$n \le k + (c - k)\lfloor k/2 \rfloor.$$

For $k = 2r$ the right side is $r(c + 2 - 2r)$; for $k = 2r + 1$ it is $r(c + 1 - 2r) + 1$. The even expression exceeds the odd one by $r - 1 \ge 0$. The cases $k = 0, 1$ are infeasible. For $c \ge 3$,

$$\max_{1 \le r \le \lfloor c/2 \rfloor} r(c + 2 - 2r) = \left\lfloor \frac{(c+2)^2}{8} \right\rfloor. \tag{E}$$

Indeed, setting $h = c + 2$ gives

$$r(h - 2r) = \frac{h^2 - (h - 4r)^2}{8}.$$

Choose an integer r nearest to $h/4$. The squared loss is 0, 1, or 4 when $h \equiv 0$, h is odd, or $h \equiv 2 \pmod 4$, respectively, giving exactly the displayed floor. Such a maximizing r lies in the required range for $c \ge 3$.

It follows that $c \geq \left\lceil \sqrt{8n} \right\rceil - 2$. Conversely set $c = \left\lceil \sqrt{8n} \right\rceil - 2$. For $n \geq 3$, $3 \leq c \leq n$ because $\sqrt{8n} \leq n + 2$. Choose a maximizing r in (E) and set $k = 2r$, $t = c - 2r$. Then $2 \leq k \leq c \leq n$ and $t \geq 0$. The inequality $n \leq r(c + 2 - 2r)$ is exactly $n - k \leq rt$. The balanced layout in Theorem B therefore attains time at most t and total cost at most c. □

This is elementary capacity arithmetic after (A) and (B). It is not offered as a new root law independent of the established throttling framework.

6 Examples and model checks

On C_5, take $B = \{0, 1, 3\}$. Every contiguous white run has length one, but there is a single arc between long blue runs, containing two white vertices. With leak $x = 0$, the newly blue sets are $\{2\}$ and then $\{4\}$, so $T(B) = 2$. Counting only the longest contiguous white run would give a false answer.

On C_{100} with $k = 14$, use seven blue pairs and white gaps of lengths $13, 13, 12, 12, 12, 12, 12$. The worst time is 13; (B) rules out time 12 for every 14-seed layout. The sum $14 + 13 = 27$ attains (D).

On C_6 with $B = \{0, 1\}$, ordinary forcing without a leak finishes in two rounds, whereas the worst one-leak time is four. Removing one initial seed instead leaves a single blue vertex and stalls. Initial-seed loss is therefore not interchangeable with this vertex-leak model.

If two or more arbitrary vertex leaks are allowed and any vertex is omitted from B, the adversary can leak both its neighbors. The omitted vertex can never be colored. Thus only the full set is robust. This is the established low-degree obstruction [1, Lemma 2.4], not an additional result.

7 Verification, provenance, and limits

The accompanying Python verifier directly simulates the literal rule for every labeled initial set and every fixed leak. It compares the results with (A), checks the adversarial endpoint, and exhaustively minimizes over all sets to check (B), (C), and (D). Its recorded author-side run is:

Quantity	Recorded value
Range	$3 \leq n \leq 16$
Initial sets	131,064
Set/leak scenarios	1,966,072
Fixed-budget entries, including infeasible sizes	147
Deadline entries	147
Mismatches	0

Construction checks and model-discrimination examples are additional and are not included in the exhaustive scenario count. The recorded environment was Python 3.12.14 on Linux, approximately 15 seconds. Verification uses the Python standard library, with no network, paid API, GPU, or cloud computation required. Windows execution has not been tested by the author-side record. Finite tests are falsification checks, not proofs for unbounded n.

User-supplied self-audit reports find no model, proof, or computational defect and report a separately written C check through $n = 22$. Its C source and raw logs were not supplied to this package, so that run is retained as a reported result, not authenticated author-side evidence. The corrected reported cumulative counts are 8,388,600 initial sets and 176,160,760 scenarios for $3 \leq n \leq 22$; the earlier approximate totals referred to $n = 21, 22$ only. Repeated reports from

the same audit process are not independent third-party confirmations. No human peer review is claimed.

The evidence archive preserves candidate v0.4 and its source-location checks. Selected source pages were checked by the author-side workflow; overall literature comparison remains partial. The unresolved question is historical originality of the exact fixed-set time statement, including equivalent formulations and direct consequences of earlier work. The publication makes the explicit elementary formula available for inspection without asserting priority or a new propagation principle.

Availability and rights. The paper source, verification materials, and audit snapshot are available in the accompanying evidence archive and at `https://github.com/yonezaemon1-hub/no41-one-leak-cycle-propagation`. Prepared with AI assistance. Copyright © 2026 Ryutaro Yonezu. All rights reserved. Research inquiries: yonezaemon1+research@gmail.com.

References

[1] J. S. Alameda, S. Dillman and F. Kenter, *Leaky forcing: a new variation of zero forcing*, Australasian Journal of Combinatorics **88**(3) (2024), 308–326. `https://ajc.maths.uq.edu.au/pdf/88/ajc_v88_p308.pdf`.

[2] J. S. Alameda, J. Kritschgau, N. Warnberg and M. Young, *On leaky forcing and resilience*, Discrete Applied Mathematics **306** (2022), 32–45. doi:10.1016/j.dam.2021.09.005. Inspected author version: `https://arxiv.org/abs/2008.06552v1`, Theorem 4.4, PDF p.13.

[3] L. Hogben, M. Huynh, N. Kingsley, S. Meyer, S. Walker and M. Young, *Propagation time for zero forcing on a graph*, Discrete Applied Mathematics **160** (2012), 1994–2005. doi:10.1016/j.dam.2012.04.003. Inspected author PDF: `https://arxiv.org/abs/1410.4191v1`, Remark 1.8, PDF p.3; the delivered PDF has a 2018 body date.

[4] S. Butler and M. Young, *Throttling zero forcing propagation speed on graphs*, Australasian Journal of Combinatorics **57** (2013), 65–71. `https://ajc.maths.uq.edu.au/pdf/57/ajc_v57_p065.pdf`.

[5] J. Carlson, L. Hogben, J. Kritschgau, K. Lorenzen, M. S. Ross, S. Selken and V. Valle Martinez, *Throttling positive semidefinite zero forcing propagation time on graphs*, Discrete Applied Mathematics **254** (2019), 33–46. doi:10.1016/j.dam.2018.06.017. Inspected author version: `https://arxiv.org/abs/1807.09404v1`, Theorem 7.1 and Lemma 7.2, PDF p.19.